

Largest Continuous Unblocked Area in Grid --- Advanced Notes (Part 1 & 2)

Problem Statement

You are given an $n \times m$ grid and k blocked cells.

Each blocked cell **blocks its entire row and its entire column**.

Find the **largest continuous unblocked rectangular area**.

Important: No two blocked cells share the same row or the same column.

Step 1: What are they actually asking?

They are **NOT** asking for:

- Number of free cells
- BFS/DFS
- Path finding

They are asking for:

Largest continuous **rectangle** consisting only of free cells.

Hidden Observation

The blocked **cell** itself is irrelevant.

Only these matter:

- Blocked row number
- Blocked column number

Example:

Blocked cell = (3,5)

Only:

- Row 3
- Column 5

become blocked.

Understanding the Grid

Example:

Rows = 5, Cols = 5

Blocked = (2,4)

Initial

```
.....  
.....  
.....  
.....  
.....
```

After blocking row 2

```
.....  
XXXXXX  
.....  
.....  
.....
```

After blocking column 4

```
...X.  
XXXXX  
...X.  
...X.  
...X.
```

Notice:

Rows and columns are blocked independently.

Brute Force Thinking

If there were no constraints:

1. Build the grid.
2. Mark blocked rows and columns.
3. Try every rectangle.
4. Check whether every cell inside is free.

5. Compute area.

Rectangle is defined by:

- top
- bottom
- left
- right

Hence four loops.

Then check every cell inside.

Two more loops.

Total complexity:

$$O(n^3 \times m^3)$$

Correct but very slow.

Why Brute Force is Slow

We repeatedly check the same cells.

Example:

Rectangle 1

```
00
00
```

Rectangle 2

```
000
000
```

The first four cells are checked again.

Repeated work.

The Optimization Question

Never ask:

How do I reduce loops?

Ask:

What repeated work am I doing?

Answer:

Repeatedly checking cells.

The Big Observation

Cells are **not randomly blocked**.

Entire rows and columns are blocked.

Therefore we don't need to reason about cells.

We can reason about:

- Rows
- Columns

This is a higher level of abstraction.

Transform the Problem

Original problem:

Largest free rectangle.

Transforms into:

Largest consecutive free rows

×

Largest consecutive free columns.

Example

Rows:

1 2 3 4 5 6 7 8

Blocked rows:

3 6

Visualization

1
2
X
4
5
X
7
8

Free groups:

- 1,2
- 4,5
- 7,8

Largest = 2

Do the same for columns.

Final Area

= maxFreeRows × maxFreeCols

Where does current - previous - 1 come from?

Blocked rows:

3

7

Rows between them:

4 5 6

Count = 3

Formula:

$7 - 3 - 1 = 3$

Why?

Because:

3 is blocked.

7 is blocked.

Don't count either.

Another example:

Blocked:

5

10

Rows:

6 7 8 9

Count

$$10 - 5 - 1 = 4$$

Why add 0 and n+1?

Suppose

Rows = 10

Blocked:

3

7

Middle gap:

$$3 \rightarrow 7$$

works.

But beginning?

Rows before 3:

1 2

There is no previous blocked row.

So create a **virtual wall** at 0.

Now

$$0 \rightarrow 3$$

Gap

$$3 - 0 - 1 = 2$$

Similarly after last blocked row:

Create virtual wall at $n+1$.

If $n=10$

wall = 11

$7 \rightarrow 11$

Gap

$11 - 7 - 1 = 3$

Now every gap uses exactly one formula.

No special cases.

This technique is called:

Sentinel / Virtual Boundary

Visual Wall Analogy

Wall	Blocked	Blocked	Wall
0	3	7	11

Free regions always exist **between two walls**.

Complete Optimization Logic

Instead of

Grid $\rightarrow$ Rectangles $\rightarrow$ Cells

Think

Blocked Rows $\rightarrow$ Largest Gap

Blocked Columns $\rightarrow$ Largest Gap

Answer

Largest Row Gap $\times$ Largest Column Gap

Pattern Recognition

If a problem contains:

- blocked positions
- occupied seats
- obstacles
- cuts
- intervals
- banned indices

Think:

Sort positions

↓

Add virtual boundaries

↓

Compute consecutive gaps

↓

Take max/min.

Interview Thinking Template

1. What changes after the operation?
 2. What am I repeatedly recomputing?
 3. Can I solve at a higher abstraction?
 4. Can I transform the problem?
 5. Can I eliminate edge cases with sentinels?
-

Cheat Sheet

Brute Force:

Grid → Every Rectangle → Every Cell

Optimized:

Blocked Rows → Max Gap

Blocked Cols → Max Gap

Answer:

$\text{MaxRowGap} \times \text{MaxColGap}$

Formula:

$\text{gap} = \text{current} - \text{previous} - 1$

Sentinels:

0

$n+1$

Reason:

One formula for beginning, middle, and end.

Part 2 - Coding Masterclass + Interview Pattern Book

These notes focus on **coding the optimized solution, code construction, interview implementation**, and the **Gap + Sentinel Pattern**.

1. From Logic → Code

Most beginners think

```
Logic
  ↓
Magic
  ↓
Code
```

This is wrong.

Experienced programmers think

```
Logic
  ↓
Sentence
  ↓
One line of code

Sentence
  ↓
One line of code

Sentence
```



One line of code

Example

Logic

Store blocked rows.



```
rows = []
```

Logic

Store blocked columns.



```
cols = []
```

Logic

Extract rows and columns.



```
for r, c in arr:  
    rows.append(r)  
    cols.append(c)
```

Every English sentence becomes **one line of code**.

2. Code Construction Step-by-Step

Step 1

Separate blocked rows and blocked columns.

```
rows = []  
cols = []  
  
for r, c in arr:
```

```
rows.append(r)
cols.append(c)
```

Step 2

Sort them.

```
rows.sort()
cols.sort()
```

Why?

Because gap calculation only works on consecutive blocked rows.

Example

```
Blocked

7
2
5
```

Cannot compute gaps.

Need

```
2
5
7
```

Step 3

Need maximum free rows.

Variables

```
maxRows = 0
prev = 0
```

Why?

prev

↓

0 (virtual wall)

Step 4

Loop over blocked rows.

Logic

Current blocked row

↓

Gap

↓

Update answer

↓

Move previous

Translate into code.

```
for row in rows:

    gap = row - prev - 1

    maxRows = max(maxRows, gap)

    prev = row
```

Step 5

Need the last gap.

Last blocked row

↓

Virtual wall (n+1)

Code

```
gap = (n + 1) - prev - 1  
  
maxRows = max(maxRows, gap)
```

Done.

Repeat the same process for columns.

3. Optimized Code Version 1 (prev)

```
def maxGap(blocked, limit):  
    blocked.sort()  
  
    prev = 0  
    ans = 0  
  
    for cur in blocked:  
        gap = cur - prev - 1  
        ans = max(ans, gap)  
        prev = cur  
  
    gap = (limit + 1) - prev - 1  
    ans = max(ans, gap)  
  
    return ans  
  
def largestUnblockedArea(n, m, arr):  
    rows = []  
    cols = []  
  
    for r, c in arr:  
        rows.append(r)  
        cols.append(c)  
  
    maxRows = maxGap(rows, n)
```

```
maxCols = maxGap(cols, m)
```

```
return maxRows * maxCols
```

4. Dry Run

Input

```
n = 8
```

Blocked Rows

```
2
```

```
5
```

Initially

```
prev = 0
```

```
ans = 0
```

Iteration 1

```
cur = 2
```

```
gap = 2 - 0 - 1 = 1
```

```
ans = 1
```

```
prev = 2
```

Iteration 2

```
cur = 5
```

```
gap = 5 - 2 - 1 = 2
```

```
ans = 2
```

```
prev = 5
```

After loop

```
gap = 9 - 5 - 1 = 3  
  
ans = 3
```

Largest consecutive free rows = 3

5. Optimized Code Version 2 (Recommended)

Instead of maintaining

```
prev
```

Insert virtual boundaries directly.

```
def maxGap(blocked, limit):  
    blocked.sort()  
    blocked = [0] + blocked + [limit + 1]  
    ans = 0  
    for i in range(1, len(blocked)):  
        gap = blocked[i] - blocked[i-1] - 1  
        ans = max(ans, gap)  
    return ans  
  
def largestUnblockedArea(n, m, arr):  
    rows = []  
    cols = []  
    for r, c in arr:  
        rows.append(r)  
        cols.append(c)  
    return maxGap(rows, n) * maxGap(cols, m)
```

6. Dry Run (Version 2)

Input

Blocked

2
5

After adding boundaries

0

2

5

9

Loop

$2 - 0 - 1 = 1$

 $5 - 2 - 1 = 2$

 $9 - 5 - 1 = 3$

Maximum

3

7. Which Version Should I Use?

Version 1

Advantages

- Doesn't create another list

Disadvantages

- Need separate last-gap calculation
-

Version 2

Advantages

- Cleaner
- Easier to understand
- No special handling
- Matches intuition perfectly
- Interview friendly

Recommendation

☒ Version 2

8. Edge Cases

No blocked rows

```
blocked = []
```

After boundaries

```
0  
n+1
```

Gap

```
n+1-0-1=n
```

Correct.

First row blocked

```
blocked=[1]
```

After boundaries

$\emptyset$
1
 $n+1$

Beginning gap

$1 - \emptyset - 1 = \emptyset$

Correct.

Last row blocked

`blocked=[n]`

After boundaries

$\emptyset$
 n
 $n+1$

Last gap

$n+1 - n - 1 = \emptyset$

Correct.

Blocked at first and last row

Example

$n=8$
`blocked=[1,8]`

Boundaries

 0 1 8 9

Gaps

 $1 - 0 - 1 = 0$ $8 - 1 - 1 = 6$ $9 - 8 - 1 = 0$

Answer

 6

9. Time Complexity

Extract rows/cols

 $O(k)$

Sorting

 $O(k \log k)$

Gap calculation

 $O(k)$

Overall

$O(k \log k)$

Space

$O(k)$

10. Interview Pattern Learned Today

Pattern Name

Largest Consecutive Gap

OR

Gap Between Sorted Positions

11. Recognition Checklist

Whenever you see

- Blocked positions
- Reserved seats
- Occupied indices
- Obstacles
- Cuts
- Walls
- Traffic lights
- Enemy rows
- Enemy columns

Immediately think

Sort

↓

Add Sentinels

↓

Find Gaps

↓

Take Maximum

12. Real World Analogy

Imagine a road.

|-----|-----|-----|

Traffic signals

5

10

18

Question

Largest road without signal?

Solution

Sort signals

↓

Add Start

↓

Add End

↓

Largest Gap

Exactly same idea.

13. Interview Thinking Template

After writing brute force ask

Question 1

What am I repeating?

Example

```
Checking cells repeatedly.
```

Question 2

Can I solve using higher abstraction?

Example

```
Cells  
↓  
Rows  
Columns
```

Question 3

Can I transform the problem?

Example

```
Largest Rectangle  
↓  
Largest Row Gap  
×  
Largest Column Gap
```

Question 4

Can I eliminate edge cases?

Example

Add 0

Add n+1

Question 5

Can one formula solve every case?

Example

gap=current-prev-1

Works for

- Beginning
- Middle
- End

14. Similar Interview Problems

This exact thinking appears in

1. Maximum consecutive ones after removing blocked positions
2. Maximum distance between traffic lights
3. Maximum empty seats
4. Largest free parking space
5. River crossing with rocks
6. Wooden stick cuts
7. Airplane runway scheduling
8. CPU time slot allocation
9. Hotel room allocation
10. Largest available memory block

15. Ultimate Pattern Template

Problem

↓

Positions matter
(Not actual values)

↓

Sort positions

↓

Add Virtual Boundaries

↓

Compute Consecutive Gaps

↓

Take Max/Min

↓

Done

16. One-Line Memory Trick

Whenever obstacles divide space, don't inspect the space—inspect the gaps between the obstacles.

17. Final Cheat Sheet

Brute Force

Grid

↓

Every Rectangle

↓

Every Cell

Optimized

Blocked Rows

↓

Largest Gap

↓

Blocked Columns

↓

Largest Gap

↓

Multiply

Formula

$$\text{gap} = \text{current} - \text{previous} - 1$$

Sentinels

0

n+1

Time Complexity

$$O(k \log k)$$

Pattern

Sort + Sentinels + Gaps

Interview Rule

Don't optimize loops. Optimize repeated work.